


**UNIVERSITÀ DEGLI STUDI DI MILANO**

Corso di Laurea in Sicurezza dei sistemi e delle reti informatiche - online

# Piattaforma per la gestione di ricette di cucina

Relazione tecnica

Progetto per l'esame di Programmazione web e mobile A.A. 2025/2026

**Studente:**
Chiara Fragomeli
Matricola: 70604A

**Docente:**
Prof. Valerio Bellandi

# Indice


**1 Introduzione** **2**

**2 Requisiti** **2**
2.1 Requisiti funzionali . . . 2
2.2 Requisiti non funzionali . . . 2

**3 Struttura e presentazione della piattaforma** **3**
3.1 Tecnologie . . . 3
3.2 Diagramma architetturale . . . 5
3.3 Backend . . . 5
3.3.1 Progettazione del database e modelli dati . . . 6
3.3.2 API REST . . . 7
3.4 Frontend . . . 10
3.4.1 Architettura e routing client-side . . . 11
3.4.2 Gestione dello stato e client HTTP . . . 12
3.4.3 Integrazione con TheMealDB . . . 12
3.4.4 Interfaccia utente e componenti custom . . . 12

**4 Prove di funzionamento** **14**
4.1 Accesso all'applicazione e navigazione . . . 14
4.2 Autenticazione e gestione del profilo . . . 17
4.3 Ricerca e consultazione delle ricette . . . 19
4.4 Gestione del ricettario personale . . . 20
4.5 Sistema di recensioni della community . . . 24
4.6 Popolamento del database e struttura dei documenti . . . 27

**A Istruzioni per l'avvio e la configurazione** **28**
A.1 Requisiti . . . 28
A.2 Installazione . . . 28
A.3 Credenziali di accesso per i test . . . 28
A.4 Struttura del progetto . . . 29

# 1 Introduzione

Il presente documento descrive il lavoro svolto per la realizzazione del progetto *Piattaforma per la gestione di ricette di cucina* nell'ambito dell'esame di Programmazione Web e Mobile. Nelle sezioni seguenti saranno analizzati i requisiti funzionali e non funzionali dell'applicazione; successivamente, saranno illustrate le scelte tecnologiche e l'architettura generale del sistema, approfondendo nel dettaglio le logiche di implementazione del backend e del frontend; infine, saranno presentate le prove di funzionamento a dimostrazione della corretta esecuzione della traccia, seguite in appendice dalle istruzioni per la configurazione e l'avvio dell'applicazione in locale.

# 2 Requisiti

## 2.1 Requisiti funzionali

Le funzionalità offerte dalla piattaforma si articolano in quattro scenari d'uso principali:

- **gestione del profilo dell'utente**: la piattaforma permette all'utente di creare un account personale attraverso un processo di registrazione che prevede l'inserimento di nome, cognome, email, password e, in modo opzionale, la selezione della categoria di ricette preferite. Una volta autenticato, l'utente può gestire le impostazioni del proprio profilo modificando il nome utente e la password, oppure può richiedere l'eliminazione definitiva dell'account ;

- **ricerca di ricette culinarie**: la piattaforma mette a disposizione di qualsiasi utente, anche non autenticato, un'interfaccia per la consultazione del catalogo THEMEALDB, un database online di ricette di cucina. L'interrogazione del database avviene tramite le API REST fornite dal servizio. L'utente ha a disposizione quattro tipologie di ricerca: per nome del piatto, per ingrediente, per lettera iniziale o per tipologia di cucina;

- **gestione di un ricettario personale**: la piattaforma consente all'utente autenticato di salvare e rimuovere le ricette cercate all'interno di un ricettario personale. Per ciascuna ricetta salvata l'utente ha inoltre la possibilità di inserire e modificare una nota testuale privata, visibile esclusivamente a lui;

- **recensioni delle ricette**: la piattaforma consente all'utente autenticato di recensire le ricette, specificando la data di preparazione e due valutazioni numeriche su scala da 1 a 5 per la difficoltà e per il gusto. Ciascun utente può lasciare al massimo una recensione per ricetta, che risulterà visibile da tutta la community.

## 2.2 Requisiti non funzionali

- **usabilità**: l'interfaccia utente implementa un'architettura *Single Page Application* (SPA) per garantire un'esperienza di navigazione fluida evitando il caricamento del browser tra le viste. Il layout è responsivo grazie alla griglia di Bootstrap 5, adattandosi a schermi desktop, tablet e smartphone. Il sistema fornisce un feedback visivo attraverso l'uso di *spinner* per le operazioni asincrone e validazione dei campi dei form in tempo reale, segnalando eventuali errori prima dell'invio dei dati al server. La ricerca delle ricette salva lo stato e i risultati correnti all'interno del `sessionStorage`, consentendo all'utente

di ritrovare l'elenco delle ricette cercate inalterato quando torna indietro dalla scheda principale di una ricetta;

- **affidabilità e robustezza**: l'applicazione implementa una gestione preventiva degli errori di rete e di parsing delle comunicazioni HTTP verso THEMEALDB e verso il backend, ricorrendo a blocchi `try/catch` e fallback per prevenire l'interruzione dell'interfaccia. Il server `Express.js` viene avviato solo dopo aver confermato l'avvenuta connessione a `MongoDB`, impedendo l'elaborazione di richieste con il database offline. L'applicazione intercetta le violazioni dei vincoli di unicità di `MongoDB`, traducendole in risposte HTTP 409;

- **efficienza e prestazioni**: i tempi di risposta sono ottimizzati limitando le chiamate superflue al server, per esempio recuperando il ricettario solo a seguito della verifica condizionale dello stato di autenticazione dell'utente, ricorrendo a costrutti sequenziali `async/await`. Nelle collezioni `Cookbook` e `Review` i metadati sono denormalizzati, per garantire una renderizzazione rapida senza dover effettuare ulteriori richieste;

- **sicurezza**: per proteggere le credenziali degli utenti, si è scelto di cifrare le password tramite l'algoritmo `bcryptjs` con un fattore di costo impostato a 12 *salt rounds*, intervenendo sia in fase di registrazione che di modifica del profilo. L'autenticazione è gestita tramite lo standard JSON Web Token (JWT), trasmesso dal client all'interno dell'header HTTP `Authorization: Bearer`. Inoltre, per tutelare l'integrità dei dati durante le operazioni di modifica o eliminazione nel database, si è implementato un filtro che associa in modo rigoroso l'id della risorsa allo `userId` estratto dal token JWT, garantendo che nessuno possa manipolare elementi creati da altri utenti. Infine, per prevenire attacchi di tipo Cross-Site Scripting (XSS), prima di iniettare dati dinamici all'interno del DOM si è provveduto a bonificare le stringhe tramite la funzione `escapeHTML()`: quest'ultima converte i caratteri speciali `<`, `>`, `&`, `"` nelle corrispondenti entità HTML, bloccando alla radice l'esecuzione di script arbitrari nel browser.

# 3 Struttura e presentazione della piattaforma

## 3.1 Tecnologie

Per la progettazione e lo sviluppo della piattaforma sono state scelte tecnologie basate sull'ecosistema JavaScript, adottando un'architettura full-stack:

- **Frontend**:
  - **HTML5 e CSS3**: struttura semantica delle viste, layout responsive e foglio di stile personalizzato (`style.css`) con palette cromatica dedicata, animazioni e carosello 3D;
  - **Bootstrap 5 e Bootstrap Icons**: framework grafico per il grid system, componenti responsivi e icone vettoriali;
  - **JavaScript (Vanilla ES6+)**: gestione del routing client-side hash-based (`router.js`), manipolazione dinamica del DOM e chiamate asincrone tramite Fetch API.

- **Backend**:
  - `Node.js`: runtime environment per l'esecuzione del codice lato server;

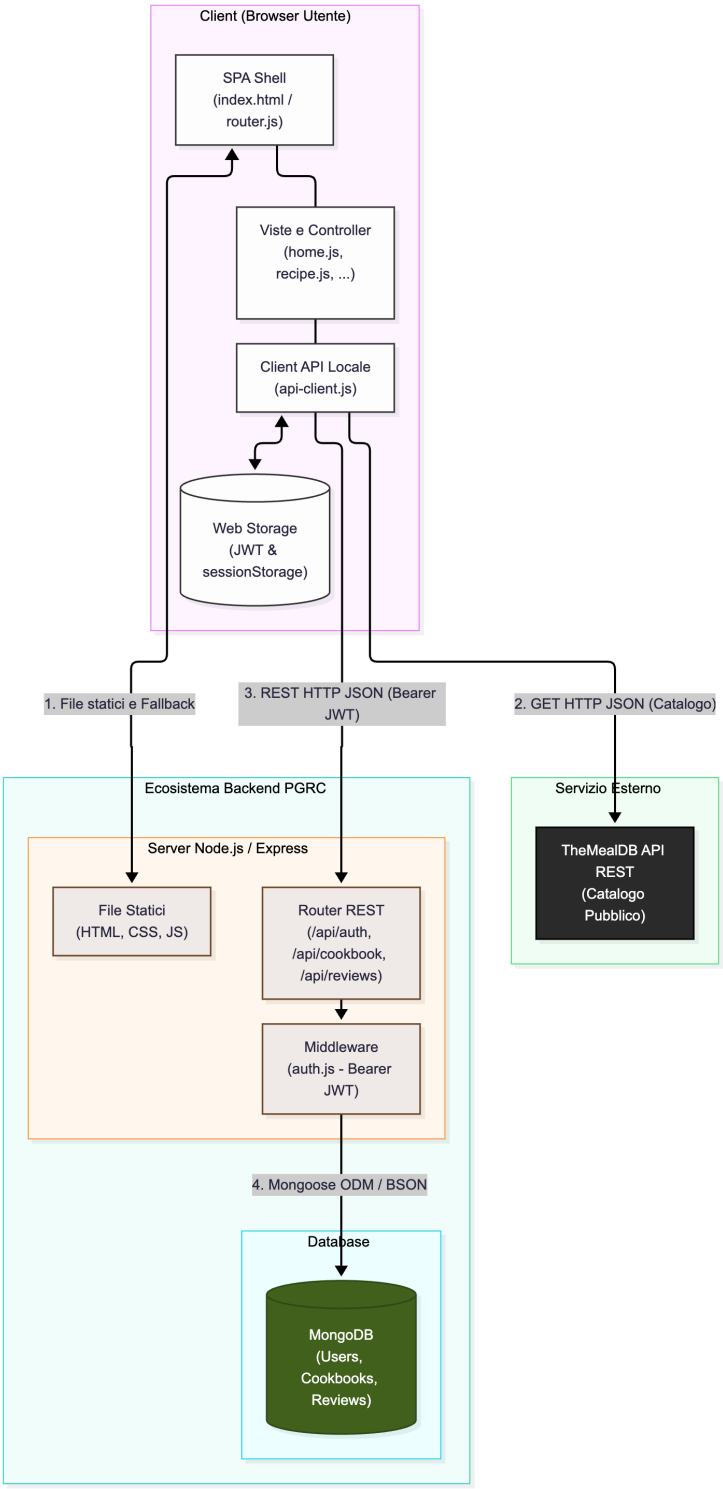
  - `Express.js`: web application framework per l'esposizione delle API RESTful, la gestione dei middleware globali e il routing statico con SPA fallback.

- **Database**:
  - `MongoDB`: database non relazionale (NoSQL) orientato ai documenti per la persistenza di utenti, ricettario e recensioni;
  - `Mongoose`: ODM (Object-Document Mapper) per modellare gli schemi, definire indici di unicità e gestire hook pre-save.

- **Dipendenze e librerie di backend**:
  - `bcryptjs`: algoritmo di hashing per la sicurezza delle password;
  - `jsonwebtoken`: emissione e verifica crittografica dei token JWT;
  - `cors`: middleware per abilitare le richieste Cross-Origin Resource Sharing;
  - `dotenv`: caricamento sicuro delle variabili d'ambiente da file `.env`.

- **Servizi di terze parti**:
  - THEMEALDB: API REST esterna e aperta per la consultazione del catalogo pubblico di ricette.

## 3.2 Diagramma architetturale



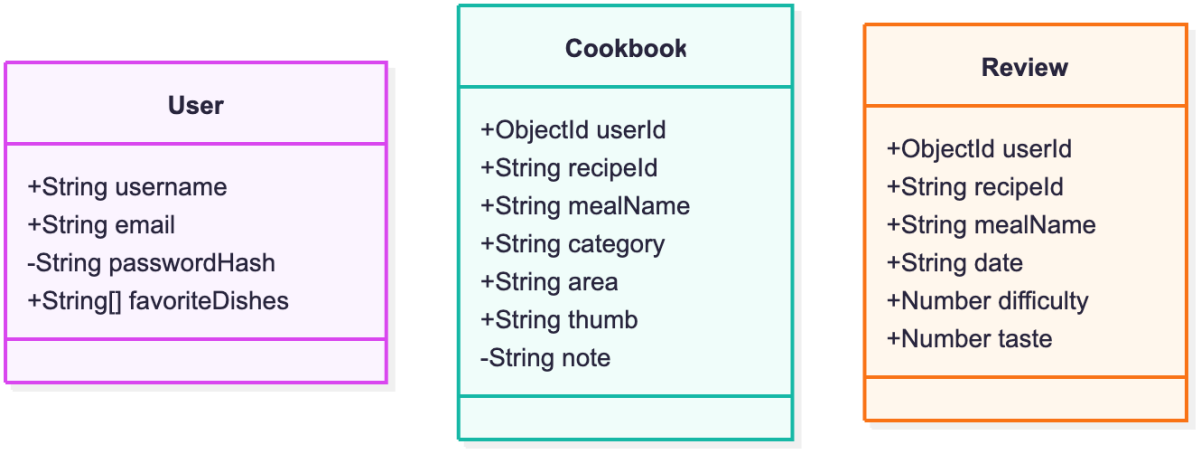

## 3.3 Backend

I requisiti di progetto prevedevano un'architettura esclusivamente client-side basata sulla Web Storage API (`localStorage`). Tuttavia, vincolare la persistenza alla memoria locale del browser introduceva limitazioni architetturali. Inoltre, gestire l'intero ciclo applicativo nel client avrebbe significato la mancanza di un ambiente di esecuzione sicuro per l'hashing delle password e per la validazione fidata delle sessioni. Per superare questi vincoli e assicurare una

netta separazione delle responsabilità, si è scelto di estendere il sistema implementando un backend in `Node.js` ed `Express.js` con API REST collegate a un database `MongoDB`.

### 3.3.1 Progettazione del database e modelli dati

Per la persistenza dei dati si è scelto di impiegare un database non relazionale tramite `MongoDB`, gestito a livello applicativo mediante l'ODM `Mongoose`. Questa scelta permette di operare con strutture dati flessibili, archiviate nativamente da `MongoDB` in formato BSON (Binary JSON) e facilmente convertibili in oggetti JavaScript per essere scambiate con il client in formato JSON. Di seguito sono descritti i modelli dati implementati:




- `User`: anagrafica e autenticazione.
  - `username` ed `email` (`String`): campi obbligatori, dotati di indici univoci per prevenire collisioni e velocizzare il login;
  - `passwordHash` (`String`): salva la password dell'utente in formato criptato. La cifratura avviene in automatico prima del salvataggio nel database grazie a un hook `pre("save")` che utilizza la libreria `bcryptjs`. Per motivi di sicurezza, questo campo è nascosto di default nelle query;
  - `favoriteDishes` (`[String]`): elenco opzionale delle categorie di cucina preferite selezionate in fase di registrazione.
- `Cookbook`: ricette salvate dagli utenti.
  - `userId` (`ObjectId`): riferimento logico all'autore;
  - `recipeId` (`String`): identificativo esterno della pietanza su THEMEALDB;
  - `note` (`String`): campo opzionale per la nota personale;
  - `mealName`, `category`, `area`, `thumb` (`String`): metadati denormalizzati della ricetta.

  L'integrità è garantita da un indice composto univoco su `(userId, recipeId)`, che impedisce salvataggi duplicati delle ricette.
- `Review`: recensioni lasciate dagli utenti.

- `userId` (`ObjectId`) e `recipeId` (`String`): chiavi esterne per l'associazione tra utente e ricetta;
- `mealName` (`String`): nome denormalizzato della pietanza;
- `date` (`String`): data di preparazione della ricetta in formato testuale;
- `difficulty` e `taste` (`Number`): metriche di giudizio su un intervallo da 1 a 5.

Un indice composto univoco limita la sottomissione a una sola recensione per ricetta da parte del medesimo utente.

**Metadati denormalizzati** Poiché i dati delle ricette sono richiamati da un'API esterna di terze parti, si è scelto di applicare una denormalizzazione: all'interno dei documenti di `Cookbook` vengono memorizzati direttamente i metadati essenziali della pietanza (`mealName`, `category`, `area`, `thumb`), mentre nelle recensioni (`Review`) viene duplicato il nome del piatto (`mealName`). Salvando questi dati direttamente nel database, si evita di dover interrogare continuamente l'API di THEMEALDB: il frontend può, quindi, caricare il ricettario e le recensioni con un'unica chiamata REST al backend, rendendo l'interfaccia molto più veloce e indipendente dai tempi di risposta del servizio esterno.

### 3.3.2 API REST

Il backend dell'applicazione è implementato in ambiente `Node.js` utilizzando il framework `Express.js`. L'architettura adotta un'impostazione modulare, grazie alla separazione della configurazione del server, della gestione del middleware di sicurezza e della definizione dei controller di routing per le risorse esposte. Il formato standard scelto per l'interscambio dei dati tra client e server è il JSON.

**Metodi HTTP** Nella progettazione degli endpoint sono state seguite le indicazioni RESTful per l'utilizzo dei metodi HTTP:

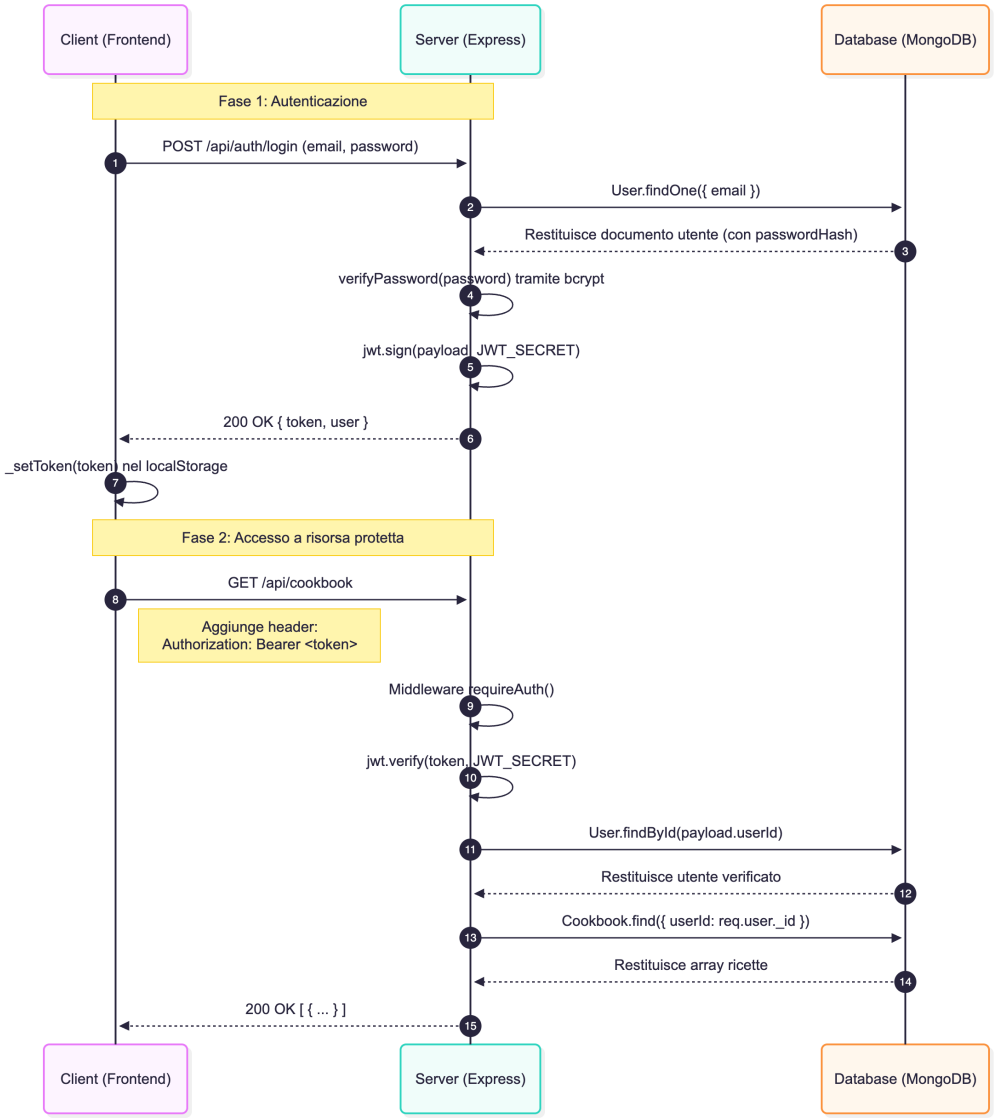
- **POST**: per la creazione di nuove risorse all'interno di una collezione, delegando al backend la generazione dell'identificativo univoco. È impiegato per la registrazione di un utente, l'autenticazione e l'inserimento di una nuova recensione;
- **PUT**: per l'aggiornamento o la sostituzione dei dati di una specifica risorsa. Nel progetto è utilizzato specificatamente per l'aggiornamento del profilo utente (username e password) tramite l'endpoint `/api/auth/me`;
- **PATCH**: per l'aggiornamento della nota privata associata al ricettario personale senza sovrascriverla interamente;
- **DELETE**: per la rimozione definitiva di una risorsa, come l'eliminazione di un account, di una ricetta salvata o di una singola recensione.

**Codici di stato HTTP** Per garantire una corretta comunicazione degli esiti delle richieste al client, il backend restituisce codici di stato HTTP standardizzati:

- **200 OK / 201 Created**: utilizzati per indicare il successo di un'operazione; il 201 è specificamente restituito dopo la creazione di una nuova risorsa (per esempio, la registrazione di un utente o l'aggiunta al ricettario);

- **400 Bad Request**: restituito quando la richiesta del client è malformata o mancano parametri obbligatori nel body (per esempio, campi vuoti durante la registrazione);
- **401 Unauthorized**: utilizzato quando l'autenticazione fallisce. Viene restituito se le credenziali di login sono errate, oppure se il token JWT fornito è mancante, malformato o scaduto;
- **404 Not Found**: impiegato per segnalare che la risorsa richiesta non esiste nel database (per esempio, tentativo di eliminare una ricetta non presente nel proprio ricettario);
- **409 Conflict**: utilizzato per gestire le violazioni dei vincoli di unicità. Viene restituito se un utente tenta di registrarsi con un'email già esistente, di salvare una ricetta già presente nel ricettario o di recensire più di una volta lo stesso piatto;
- **500 Internal Server Error**: inviato in caso di eccezioni non gestite o errori imprevisti del database.

**Sistema di autenticazione** L'autenticazione è gestita da un sistema basato sui token JWT: invece di salvare le sessioni sul server, tutte le informazioni dell'utente sono impacchettate direttamente nel token. Per evitare che qualcuno lo modifichi, il server firma crittograficamente il token usando una chiave segreta. Al momento dell'autenticazione o della registrazione, il server genera un access token con una validità di 7 giorni e lo restituisce al client. Il frontend si occupa di memorizzare il token nel `localStorage` e di allegarlo a ogni successiva richiesta HTTP protetta attraverso l'header `Authorization` nel formato `Bearer <token>`. Il middleware intercetta le richieste, verifica la firma e la validità temporale del token e, in caso di successo, estrae i dati dell'utente dal database rendendoli disponibili per i successivi livelli di logica applicativa.




**Endpoint** Le rotte dell'API sono suddivise in tre moduli:

- **autenticazione e profilo** (`/api/auth`)
  - `POST /register`: riceve i dati utente, effettua i controlli di unicità sull'email, esegue l'hashing della password tramite `bcryptjs` e salva il nuovo utente nel database, restituendo immediatamente il token JWT;
  - `POST /login`: verifica la corrispondenza tra l'email fornita e l'hash della password salvata nel database, restituendo il token di accesso in caso di successo;
  - `GET /me`: restituisce i dati dell'utente loggato. Questo endpoint serve al frontend per ripristinare la sessione in automatico e recuperare i dati del profilo ogni volta che si aggiorna la pagina;
  - `PUT /me`: permette all'utente di aggiornare il nome utente o la password;
  - `DELETE /me`: esegue l'eliminazione definitiva dell'account. Per mantenere l'integrità del database, implementa una cascade delete parallela, rimuovendo anche l'intero ricettario e tutte le recensioni associate all'utente.

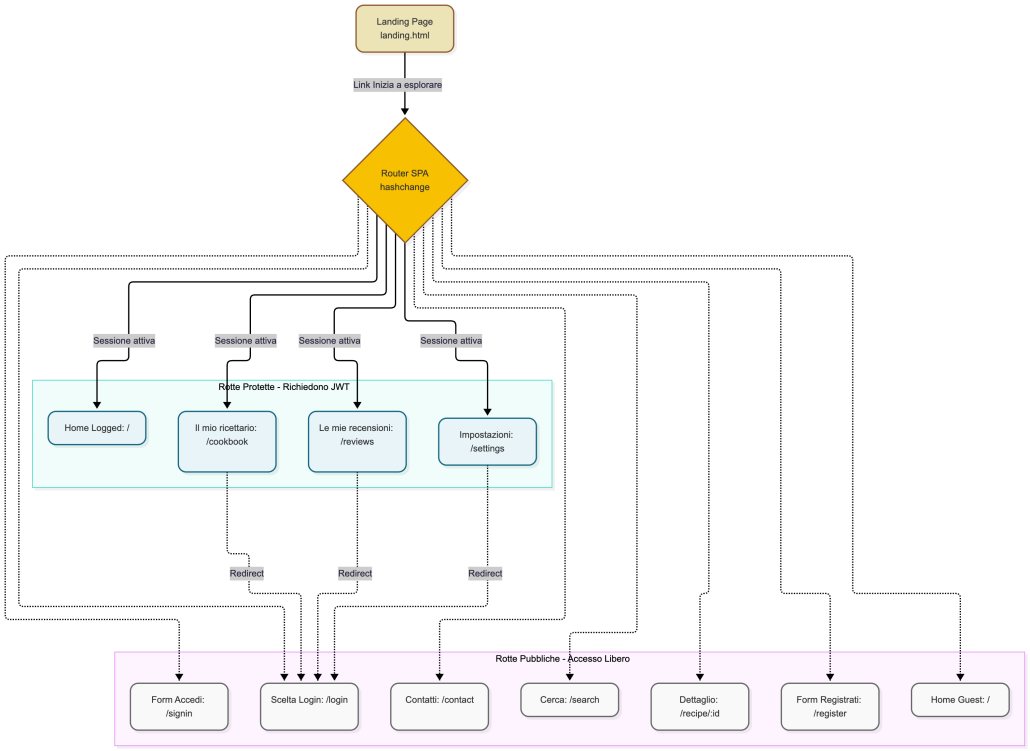
- **ricettario personale** (`/api/cookbook`):
  - `GET /`: recupera tutte le ricette salvate dall'utente corrente in ordine cronologico decrescente;
  - `POST /`: aggiunge una ricetta al ricettario personale;
  - `PATCH /:recipeId`: permette la modifica parziale della voce nel ricettario per l'aggiornamento della nota personale;
  - `DELETE /:recipeId`: rimuove dal ricettario la ricetta selezionata.
- **recensioni** (`/api/reviews`):
  - `GET /`: restituisce tutte le recensioni salvate. Attraverso una query string opzionale è possibile filtrare i risultati per mostrare solo le recensioni di una specifica ricetta. Per recuperare il nome dell'autore in modo efficiente, è sfruttato il metodo `populate` di `Mongoose`, che esegue una sorta di JOIN con la collezione degli utenti;
  - `GET /mine`: restituisce unicamente le recensioni scritte dall'utente autenticato;
  - `POST /`: inserisce una nuova recensione. Un indice univoco previene inserimenti multipli per la stessa combinazione utente-ricetta;
  - `DELETE /:id`: elimina una recensione. Per sicurezza, il server cerca la recensione nel database e verifica che l'autore corrisponda all'utente loggato.

**Configurazione del server e gestione della SPA** Nel progetto, il server `Node.js` non si occupa solo delle API REST, ma serve direttamente anche i file statici del frontend. Dal punto di vista della configurazione, per evitare conflitti tra il router lato client e il backend, la gestione delle rotte non valide per le chiamate API è stata nettamente separata: le richieste errate verso `/api/*` restituiscono un errore 404 in formato JSON, mentre qualsiasi richiesta client-side non intercettata dalle route API (wildcard `*`) rimanda sempre al file `index.html`, consentendo al router del frontend di gestire l'instradamento. Inoltre, il middleware `cors` è usato per autorizzare le richieste esterne e il modulo `dotenv` è impiegato per caricare le variabili d'ambiente in modo sicuro.

## 3.4 Frontend

Per lo sviluppo dell'interfaccia utente si è scelto il paradigma della *Single Page Application* (SPA), in modo da offrire una navigazione fluida ed evitare ricaricamenti dell'intera pagina. Per mantenere il progetto leggero e avere il controllo totale sull'architettura del codice, tutto il frontend è stato realizzato esclusivamente con HTML5, CSS3 e JavaScript "Vanilla" (ES6+). Infine, per gestire in modo semplice la responsività del layout sui vari dispositivi mobili e avere a disposizione componenti grafici già pronti, è stata utilizzata la libreria Bootstrap 5.

### 3.4.1 Architettura e routing client-side




La gestione della navigazione è affidata a un router custom implementato lato client e basato sull'hash dell'URL. Ad ogni variazione dell'hash (ad esempio `#/cookbook` o `#/recipe/53116`), un *event listener* in ascolto sull'evento `hashchange` richiama la funzione principale di rendering.

Il router si occupa di:

- **validazione e accesso**: verificare se la rotta richiesta appartiene alla lista delle rotte protette. In assenza di una sessione attiva, l'utente viene automaticamente reindirizzato alla pagina di autenticazione;
- **risoluzione dinamica**: estrarre il percorso e gli eventuali parametri per mappare la richiesta alla corretta funzione generatrice della vista;
- **aggiornamento dell'interfaccia**: inserire dinamicamente il nuovo contenuto HTML nel contenitore principale e aggiornare contemporaneamente la barra di navigazione.

**Pattern view e init** Per garantire un feedback visivo immediato e ottimizzare l'esperienza utente, l'applicazione adotta un pattern architetturale in due fasi per ogni pagina:

1. **Fase di View (Sincrona)**: una funzione restituisce immediatamente lo scheletro HTML della pagina, spesso includendo uno *spinner*;
2. **Fase di Init (Asincrona)**: terminato l'inserimento nel DOM, il router invoca una funzione asincrona che si occupa di effettuare le chiamate API necessarie e popolare i contenitori vuoti con i dati definitivi.

### 3.4.2 Gestione dello stato e client HTTP

Lo stato applicativo è gestito sfruttando la Web Storage API. I dati persistenti, come il token JWT e i dati dell'utente, sono salvati nel `localStorage`, permettendo di mantenere l'autenticazione anche in seguito alla chiusura del browser. Informazioni effimere, come i parametri dell'ultima ricerca effettuata, sono invece mantenuti nel `sessionStorage` per garantire la continuità della navigazione durante la sessione corrente, ma non oltre.

Tutte le comunicazioni con il backend sono state raggruppate in un unico file (`api-client.js`). Il core di questo modulo è una funzione asincrona basata sulla `Fetch API`, che ha il compito di:

- allegare automaticamente l'header `Authorization: Bearer <token>` a ogni richiesta in uscita, prelevando il JWT dallo storage locale;
- serializzare i payload in JSON e specificare il corretto `Content-Type`;
- intercettare le risposte, distinguendo eventuali errori HTTP da cadute di rete, restituendo al chiamante un formato dati standardizzato o sollevando eccezioni controllate.

### 3.4.3 Integrazione con TheMealDB

Per fornire agli utenti il catalogo delle ricette, l'applicazione si integra con l'API REST pubblica di THEMEALDB. Questa integrazione è gestita interamente lato client tramite un modulo isolato, per garantire una netta separazione tra le comunicazioni verso l'esterno e quelle dirette al backend proprietario.

Poiché è utilizzata la versione pubblica dell'API, le richieste non necessitano di autenticazione. Le chiamate di rete sono implementate sfruttando la Fetch API e i costrutti `async/await`, e inglobate in blocchi `try/catch` per restituire array vuoti o valori nulli in caso di errore e quindi prevenire interruzioni dell'interfaccia utente.

Tra le principali logiche di integrazione implementate ci sono:

- **normalizzazione dei dati**: l'API di THEMEALDB restituisce i dati in modo non strutturato. Per rendere i dati più facili da usare nell'interfaccia, è stata scritta la funzione `extractIngredients`, che raggruppa i campi in un array e scarta i valori vuoti o nulli;
- **fetching parallelo**: per riempire il carosello della homepage servono diverse ricette casuali, ma l'endpoint (`/random.php`) ne restituisce solo una alla volta. Invece di scaricarle una dopo l'altra rallentando la pagina, si è scelto di usare `Promise.all` per inviare tutte le richieste in parallelo, riducendo notevolmente i tempi di caricamento complessivi.

### 3.4.4 Interfaccia utente e componenti custom

L'interfaccia utente è stata progettata usando il sistema a griglia di Bootstrap 5 per la reattività del layout. Tuttavia, l'implementazione dei componenti più complessi e interattivi è stata realizzata in JavaScript puro, per non dipendere da plugin esterni.

**Componenti grafici chiave** Per l'interfaccia utente si è scelto di progettare componenti visivi coerenti e responsivi, in modo da offrire un'esperienza di navigazione fluida su qualsiasi dispositivo.

**Navbar dinamica:** la barra di navigazione è stata strutturata utilizzando il tag semantico `<nav class="navbar navbar-dark">`, personalizzandone lo sfondo tramite la variabile CSS custom `var(--green)`. Per gestirne la visibilità in base allo stato della sessione, si sono definiti due contenitori distinti (`#nav-guest` e `#nav-user`). Tramite JavaScript si interviene direttamente sullo stile, rimuovendo o aggiungendo la proprietà `display: none` per alternare i link di registrazione e il menu a tendina del profilo utente (si veda 4.1).

**Card delle ricette:** per la presentazione delle ricette si è sfruttato il sistema a griglia di Bootstrap con le classi `row-cols-1 row-cols-md-3`. Le schede utilizzano la classe `.h-100` per il corretto allineamento verticale, mentre per le immagini è stata definita la classe CSS `.card-recipe-img`, che impone un'altezza fissa di 200px unitamente alla proprietà `object-fit: cover` per evitarne la distorsione. Le valutazioni grafiche, infine, sono state costruite iniettando icone vettoriali (`<i class="bi bi-star-fill">`) stilizzate con il colore custom `var(--yellow)` (si veda 4.3).

**Feedback visivo e pop-up:** per la validazione dei form (come login o registrazione) si è preferito un approccio non invasivo basato sulla manipolazione del DOM: i messaggi di errore vengono mostrati rimuovendo la classe `.d-none` dai div di alert, mentre agli elementi `<input>` viene applicata la classe `.is-invalid` per evidenziarne i bordi in rosso. Per le notifiche istantanee e le azioni critiche si è scelto di fare ricorso alle finestre di dialogo native del browser: si richiama un `alert()` di sistema per confermare il salvataggio della nota privata e un `confirm()` bloccante per accertarsi dell'effettiva volontà dell'utente prima di procedere all'eliminazione irreversibile dell'account (si veda 4.2).

**Carosello 3D** Nella dashboard principale è presente un carosello 3D per la presentazione visiva delle ricette. Per permettere uno scorrimento infinito delle slide, è stata usata l'aritmetica modulare sull'indice dell'array: in questo modo, una volta arrivati all'ultima ricetta, si riparte automaticamente dalla prima. Per rendere fluide le animazioni di transizione, si è sfruttato un doppio `requestAnimationFrame`, che aggiorna l'opacità degli elementi sincronizzandosi con il browser. Infine, l'avanzamento automatico è gestito con un `setInterval`; per evitare problemi di prestazioni o perdite di memoria, ci si assicura di fermare sempre il timer tramite al momento del cambio pagina.

**Prevenzione vulnerabilità XSS (Cross-Site Scripting)** Poiché nel progetto il DOM viene aggiornato spesso inserendo template tramite la proprietà `innerHTML`, si è prestata molta attenzione alla sicurezza. Per evitare attacchi XSS, tutti i dati provenienti dall'esterno (come i nomi utente letti dal database o i dati scaricati dalle API) sono filtrati tramite una funzione di bonifica (`escapeHTML`). Questa funzione converte i caratteri speciali (&, <, >, ") in entità HTML innocue, bloccando sul nascere eventuali tentativi di code injection. Inoltre, nei punti in cui si visualizzano i testi scritti direttamente dall'utente (come gli appunti personali o i campi dei form), si è scelto di usare `textContent` o di aggiornare direttamente la proprietà `value` dei campi. In questo modo, si forza il browser a leggere i dati come semplice testo, evitando che vengano interpretati come codice HTML.

# 4 Prove di funzionamento

Al fine di validare il corretto funzionamento della piattaforma e dimostrare il soddisfacimento dei requisiti funzionali descritti in fase di analisi, vengono di seguito riportati i principali scenari d'uso del sistema, documentati attraverso una serie di schermate acquisite durante l'esecuzione dell'applicazione.

## 4.1 Accesso all'applicazione e navigazione

L'ingresso alla piattaforma avviene tramite una *landing page* statica di presentazione. Una volta avviata la SPA, la navbar adatta dinamicamente il proprio contenuto e la propria visibilità in base allo stato della sessione e alla rotta corrente.




Landing page.



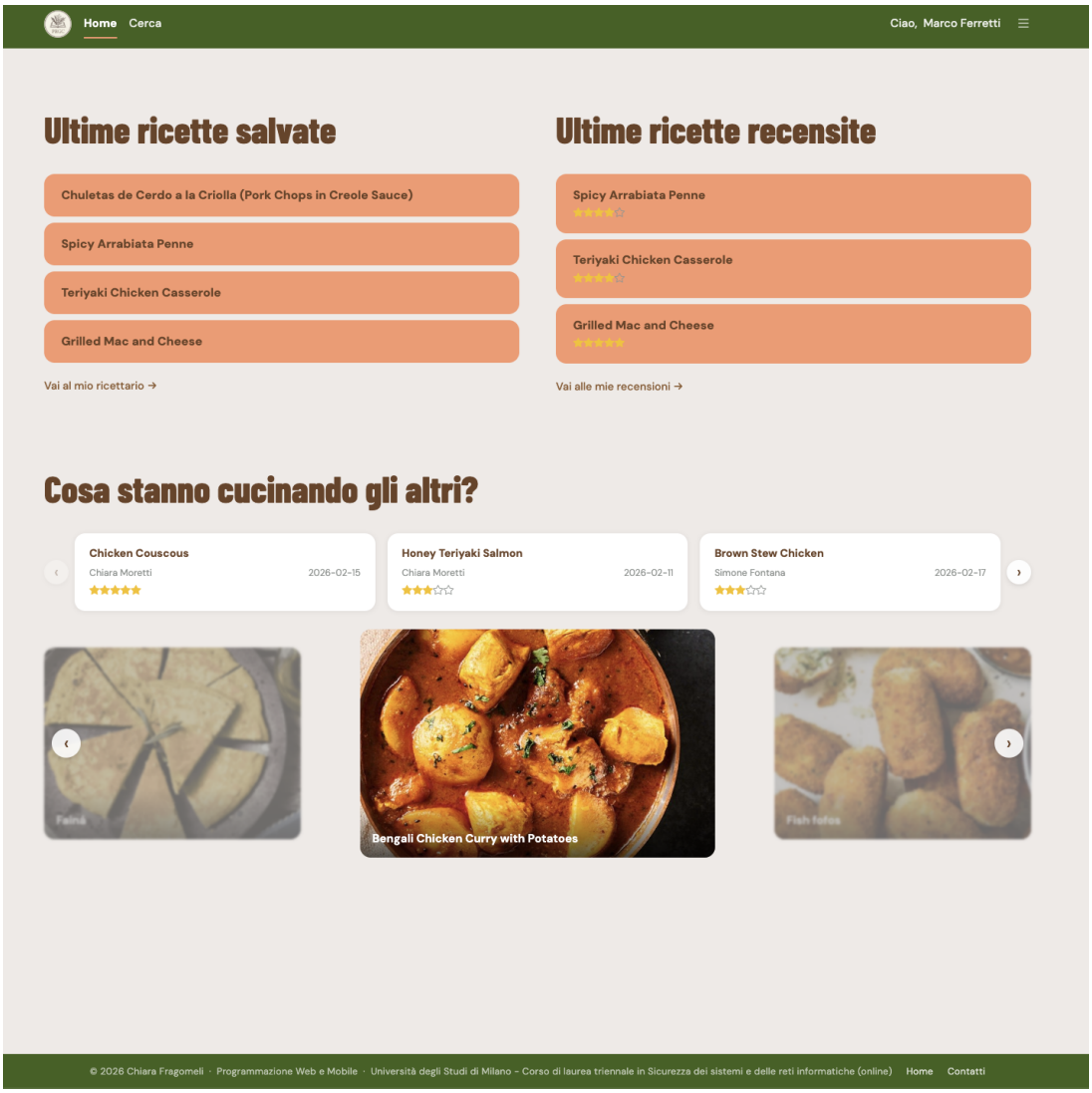

Navbar utente ospite.




Navbar utente autenticato.

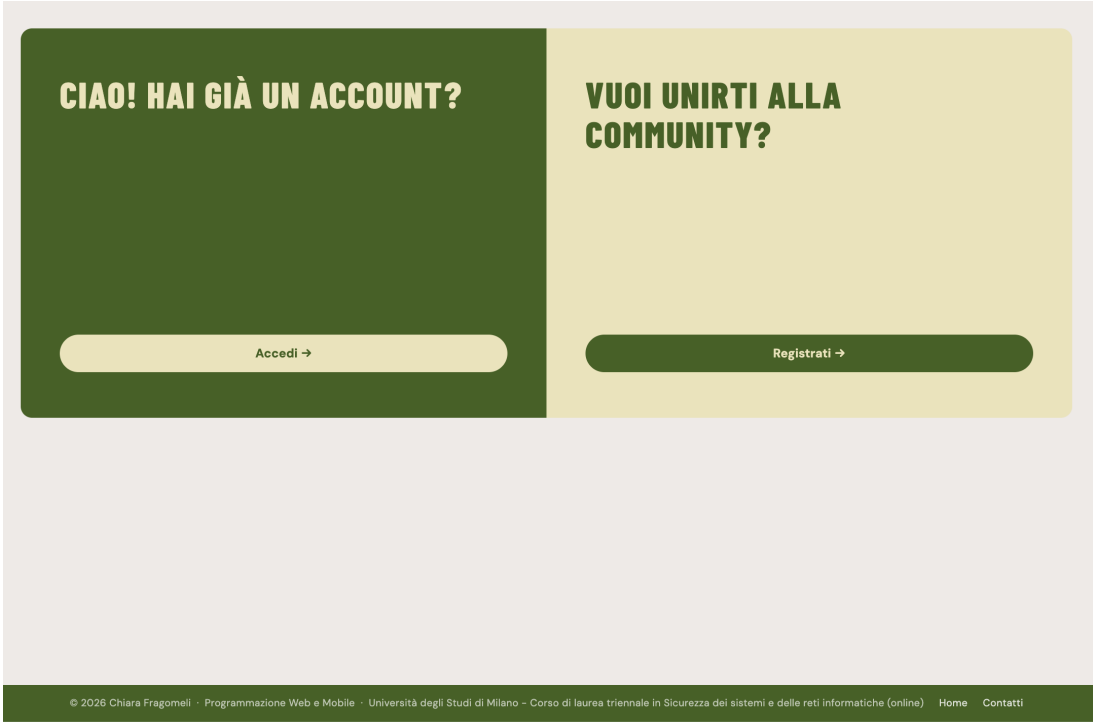



Homepage per utente ospite.

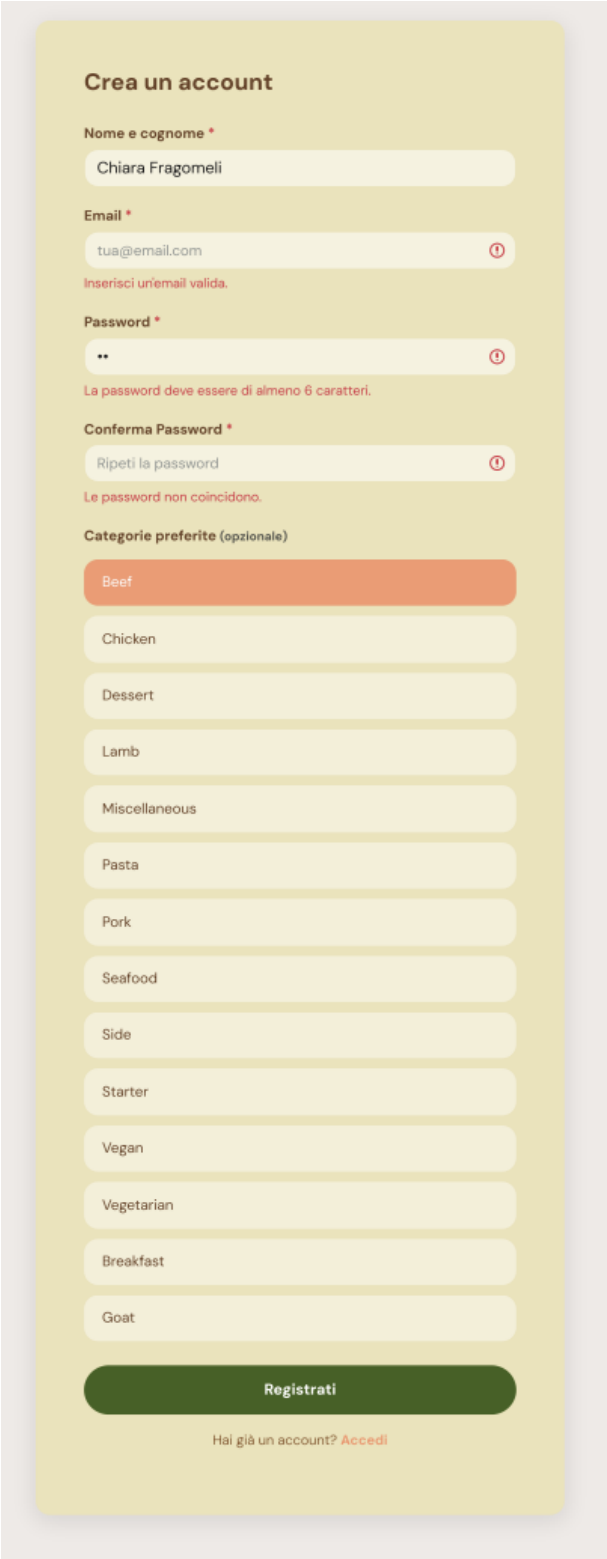



Homepage per utente autenticato.

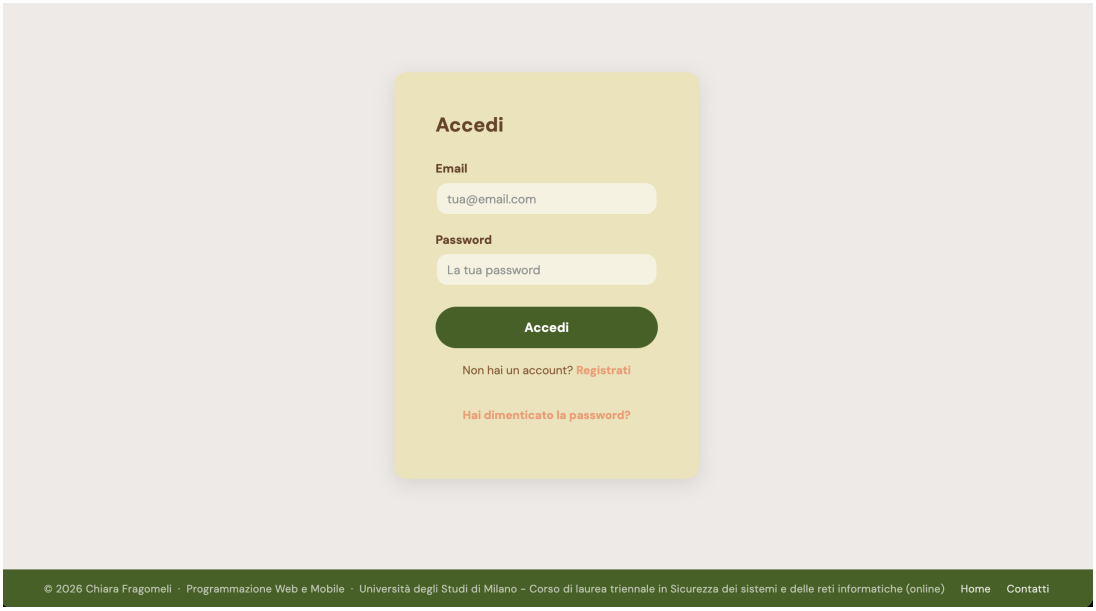
## 4.2 Autenticazione e gestione del profilo



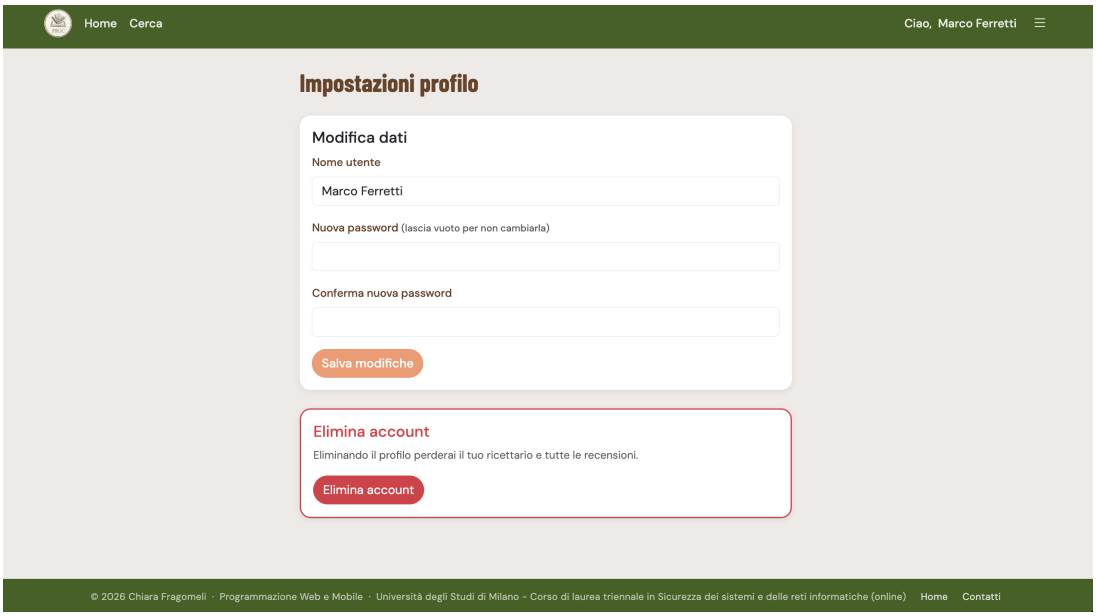

Interfaccia per l'accesso o la registrazione alla piattaforma.




Form di registrazione.

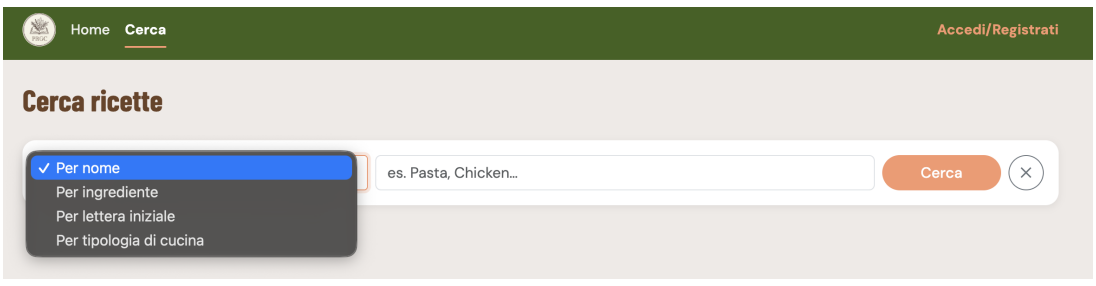



Form di autenticazione.




Impostazioni del profilo utente.

## 4.3 Ricerca e consultazione delle ricette




Interfaccia di ricerca delle ricette.

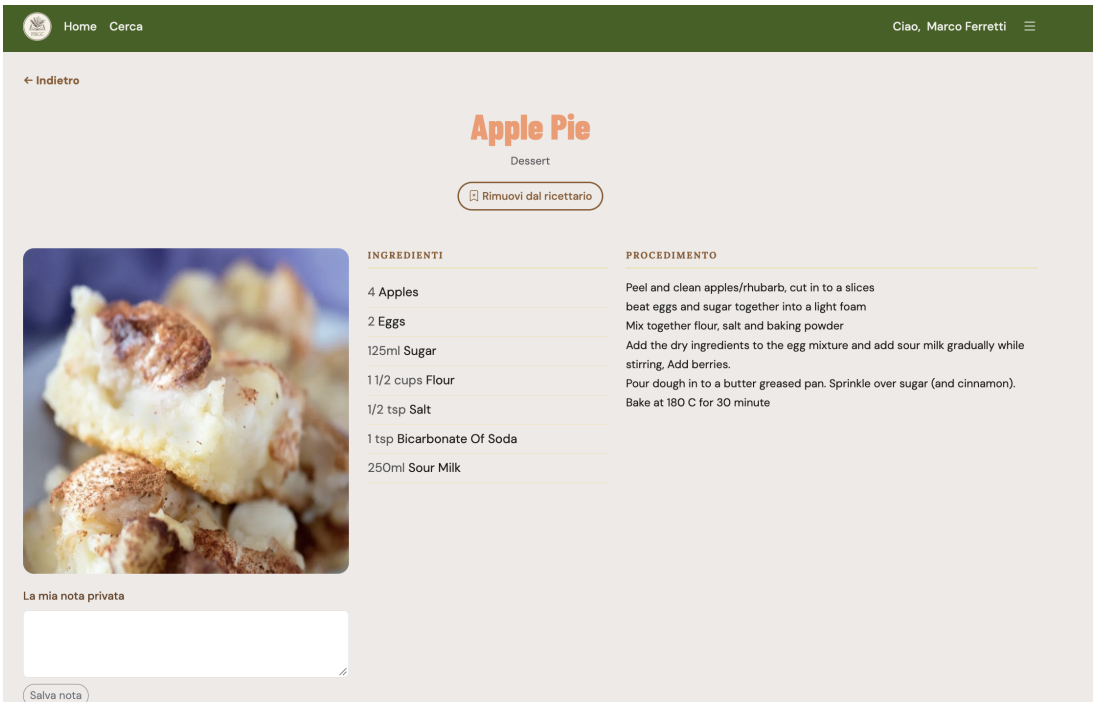



Pagina di dettaglio di una ricetta.

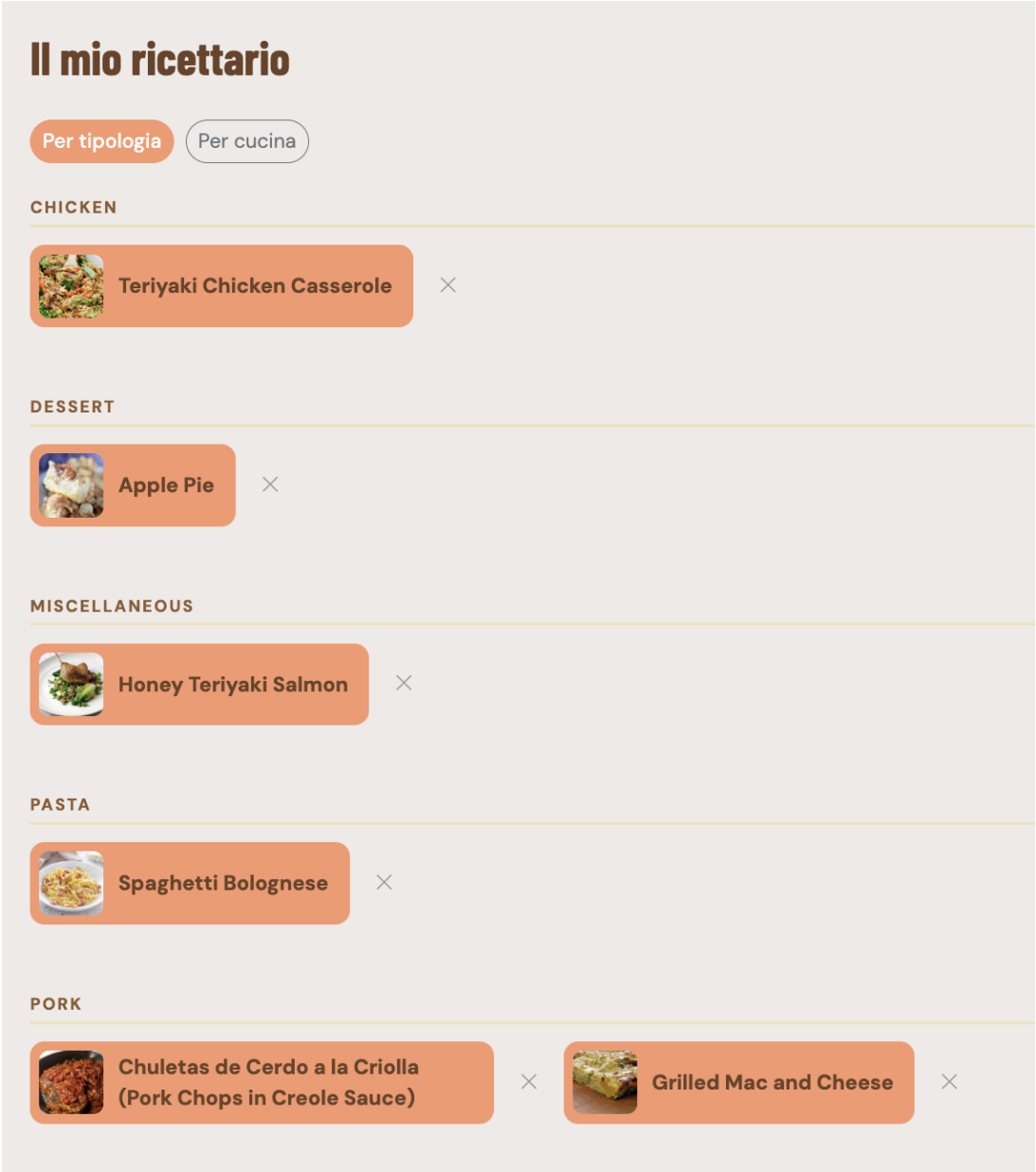
## 4.4 Gestione del ricettario personale



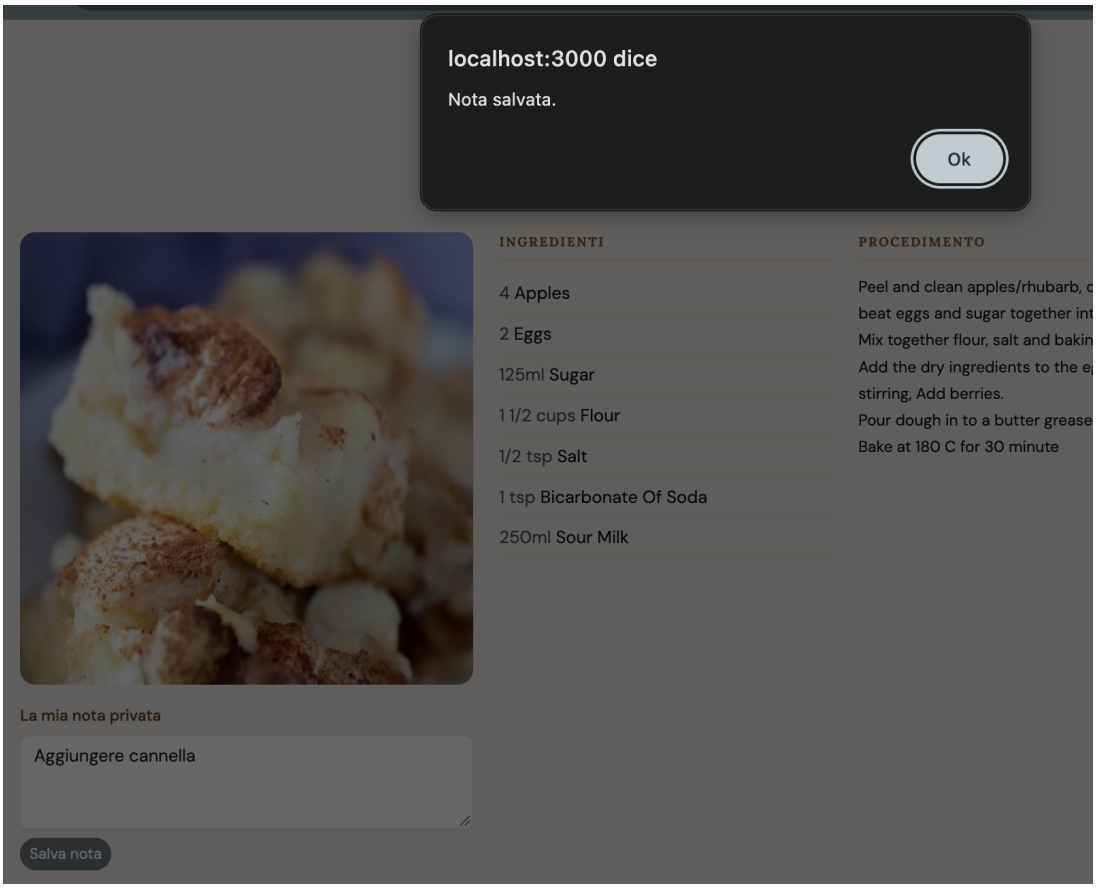

Aggiunta al ricettario.




Rimozione dal ricettario.




Ricettario personale.

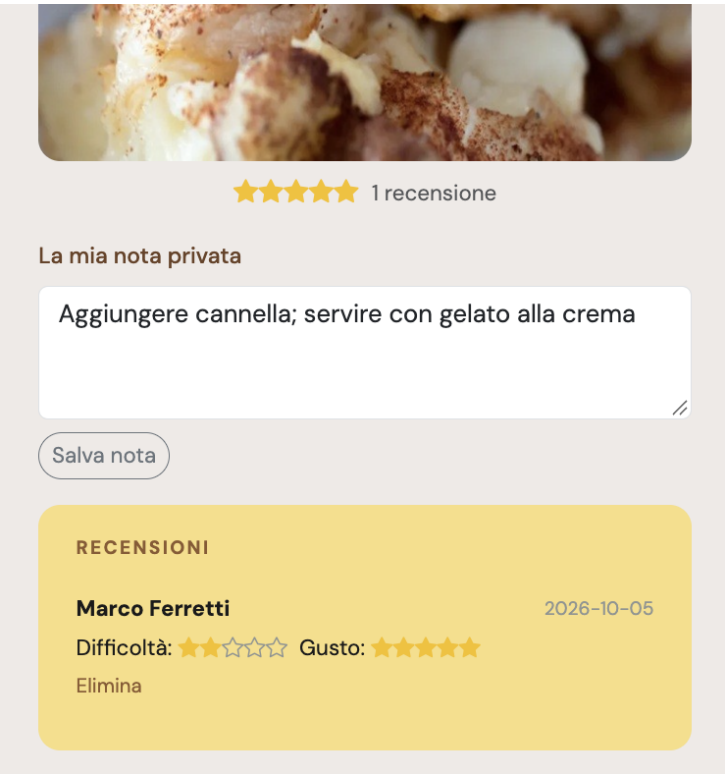



Salvataggio di una nota personale.

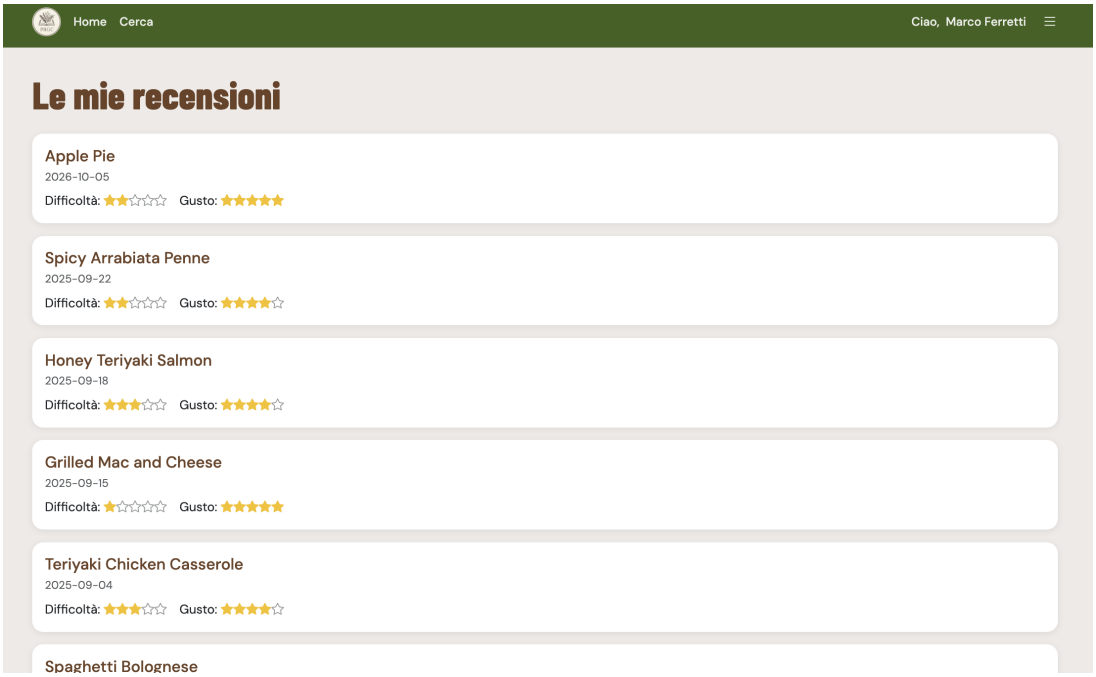
## 4.5 Sistema di recensioni della community



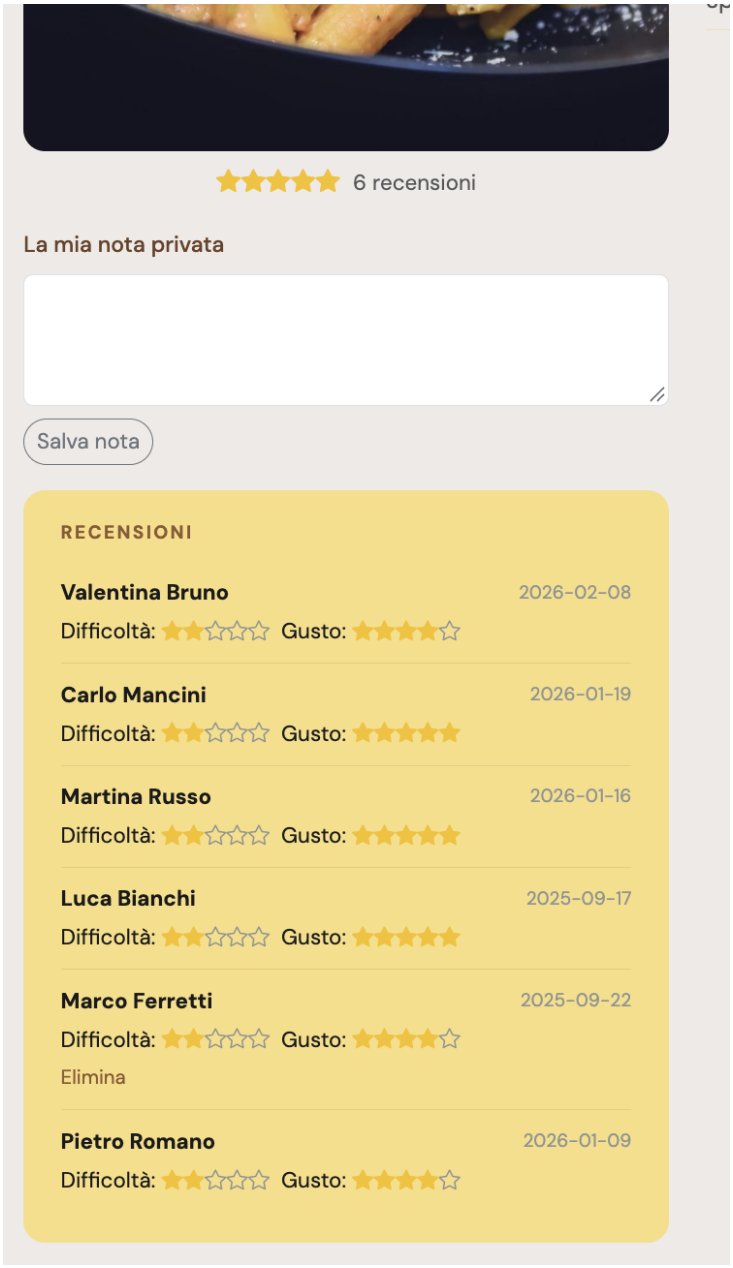

Inserimento di una recensione.




Inserimento di una recensione.

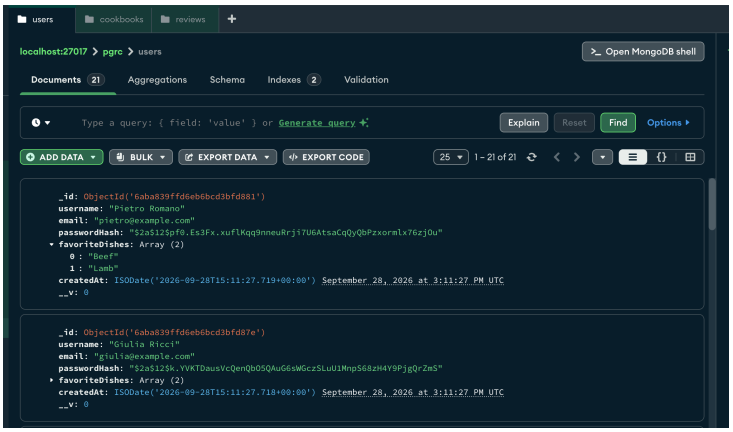



Riepilogo delle recensioni lasciate dall'utente loggato.

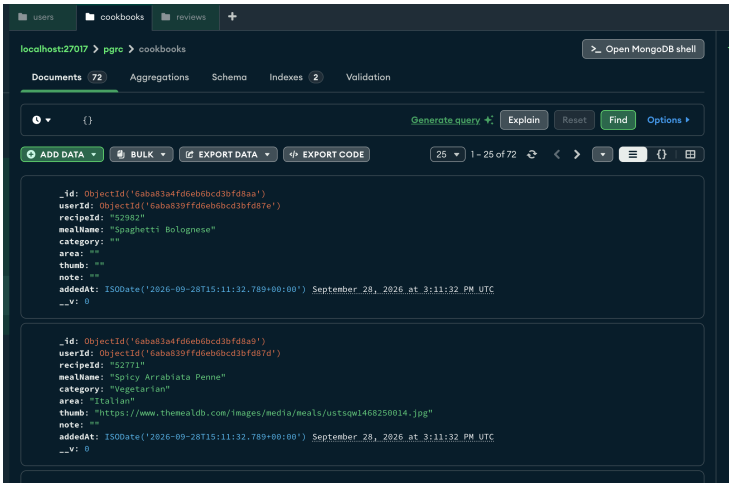



Visualizzazione delle recensioni lasciate da altri utenti.

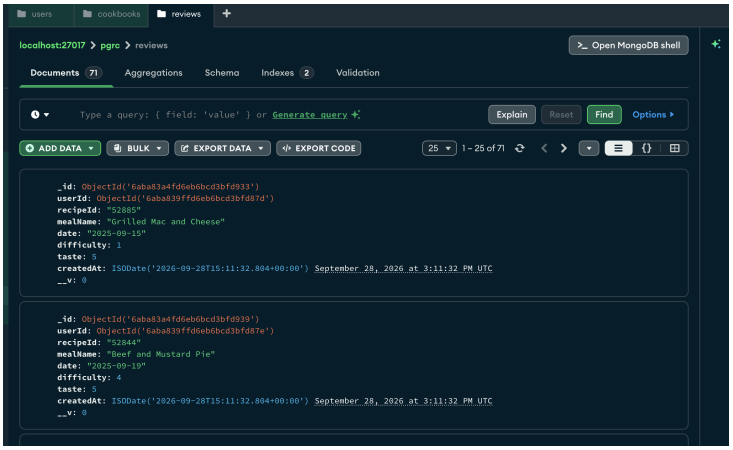
## 4.6 Popolamento del database e struttura dei documenti




Documento `users`.




Documento `cookbooks`.




Documento `reviews`.

# A Istruzioni per l'avvio e la configurazione

## A.1 Requisiti

Per eseguire correttamente l'applicazione in locale è necessario installare:

- **Node.js**: versione 18 o superiore.
- **MongoDB Community**: versione 6 o superiore.

## A.2 Installazione

- Estrarre la cartella del progetto in una posizione a scelta;
- Avviare MongoDB (deve essere in esecuzione prima di avviare il server):
  - macOS: `brew services start mongodb-community`
  - Windows: `net start MongoDB`
  - Linux: `sudo systemctl start mongod`
- Aprire il terminale nella cartella del progetto e installare le dipendenze: `npm install` ;
- Il file di configurazione `.env` è già incluso nella cartella con i parametri predefiniti:
  `MONGO_URI=mongodb://localhost:27017/pgrc`
  `JWT_SECRET=pgrc_super_secret_key_2026`
  `PORT=3000` ;
- Popolare il database con i dati di esempio: `npm run seed` ;
- Avviare il server: `npm start` ;
- Aprire il browser e navigare all'indirizzo: `http://localhost:3000` ;

## A.3 Credenziali di accesso per i test

Per testare le funzionalità riservate agli utenti autenticati è possibile registrare un nuovo account oppure utilizzare uno dei 20 profili generati dallo script di seeding (ad esempio `marco@example.com` o `giulia@example.com`). Tutti gli account preimpostati condividono la seguente password: **`password123`**.

## A.4 Struttura del progetto

```
PGRC_2026/
├── server.js ..... entry point del server Express.js
├── .env ..... configurazione ambiente (preconfigurato)
├── package.json ..... dipendenze Node.js
├── seed-mongo.js ..... script di popolamento database
├── routes/ ..... endpoint API REST
│   ├── auth.js ..... /api/auth (login, registrazione, profilo)
│   ├── cookbook.js ..... /api/cookbook (ricettario personale)
│   └── reviews.js ..... /api/reviews (recensioni)
├── models/ ..... schemi Mongoose (MongoDB)
│   ├── User.js
│   ├── Cookbook.js
│   └── Review.js
├── middleware/
│   └── auth.js ..... verifica token JWT
├── js/ ..... JavaScript lato browser
│   ├── api-client.js ..... chiamate al backend
│   ├── api.js ..... chiamate a TheMealDB
│   ├── router.js ..... router client-side
│   └── views/ ..... viste della SPA
├── index.html ..... applicazione principale
├── landing.html ..... pagina di presentazione statica
└── style.css ..... stili globali
```